

User Stories, Acceptance Criteria & Edge Cases

MVP Scope, Functional Tickets & Exception Handling Specifications

Version: 1.0.0 **Date:** July 2026 **Status:** Approved Specification **Complement to:** BRD, ERD & TDD

1. MVP Scope Definition

This section defines the mandatory scope for the initial Minimum Viable Product (MVP) release versus deferred features:

Domain	In-Scope for MVP	Out-of-Scope (Deferred)
Tree Rendering	Infinite Canvas with pan/zoom, node styling for living/deceased/pending status.	High-resolution PDF tree export, Sunburst interactive chart view.
Data Entry	Directional (+) buttons for direct relations (Parent, Child, Spouse) with quick form modal.	Audio voice notes, historical document archives, and photo albums.
Data Quality	Real-time match detection banner during input + Side-by-side merge tool for stewards.	AI face recognition in historical photos and GEDCOM file import/export.
Auth & Review	Authentication, profile claiming ("This is Me"), and steward review dashboard (<code>PENDING</code> → <code>VERIFIED</code>).	Complex kinship relationship calculator and automated DNA matching integration.

2. User Stories & Acceptance Criteria

US-01: User Registration & Profile Claiming ("This is Me") | Role: End User (USR)

As a new family member, **I want to** search for my name in the tree and claim my profile node, **so that** I can manage my direct family lineage.

Acceptance Criteria:

- On onboarding, users can search existing public tree nodes by full name or birth year.
- Selecting a node opens a modal to submit a claim request with proof note.
- The request transitions to `CLAIM_PENDING` and notifies the assigned Branch Steward.
- Until approved, the user can browse the tree in read-only mode for that node.

US-02: Add Direct Relation via Contextual (+) Buttons | Role: End User (USR)

As an authenticated user, **I want to** click (+) buttons around a node, **so that** I can quickly add direct relatives.

Acceptance Criteria:

- Clicking a node displays 4 contextual (+) buttons: Top (Parent), Bottom (Child), Right/Left (Spouse).
- Selecting a (+) button opens a modal pre-setting the gender based on relation choice (e.g., "Father" auto-selects "Male").
- On submit, the node appears on canvas with a dashed border and "Pending Review" badge (**PENDING**).

US-03: Real-time Match Detection During Input | Role: End User (USR)

As a data contributor, **I want** the system to alert me if a person already exists, **so that** I don't duplicate nodes.

Acceptance Criteria:

- A debounced background query (300ms) runs as first name and birth year are typed.
- If composite match score $\geq 75\%$, a banner presents potential duplicate nodes.
- User can click "**Link to Existing Person**" or "**Create New Person**".

US-04: Steward Verification Dashboard | Role: Branch Steward (REV)

As a Branch Steward, **I want** a dashboard to review pending requests, **so that** I can verify relations for my branch.

Acceptance Criteria:

- Notifications are dispatched exclusively to stewards assigned to the target branch.
- Dashboard displays submitter identity, proposed relationship, and connected nodes.
- Clicking "**Approve**" updates relationship to **VERIFIED**, broadcasting live canvas updates via WebSockets.

3. Edge Cases & Exception Handling

⚠ Edge Case 1: Circular Relationship Prevention

Scenario: A user attempts to add Person X as "Parent" of Person Y, while Person X is already registered as a "Child" of Person Y.

Engine Solution: Before committing, Backend runs a recursive CTE upward traversal. If **Target_ID** exists in the descendant path of **Source_ID**, the API rejects the request with HTTP **422 Unprocessable Entity** ("Circular kinship loop detected").

⚠ Edge Case 2: Concurrent Merge Conflicts

Scenario: Two stewards simultaneously click "Merge" on the same duplicate person record.

Engine Solution: Transaction uses `SELECT FOR UPDATE` pessimistic locking on target person records in PostgreSQL. The first transaction succeeds, while the second receives an alert ("Record already merged").

⚠ Edge Case 3: Unknown or Missing Ancestors

Scenario: A user wants to link cousins, but the exact name of their deceased intermediate uncle/grandparent is unknown.

Engine Solution: System supports placeholder nodes (`is_placeholder = true`) to preserve structural graph integrity until exact genealogical names are documented.

⚠ Edge Case 4: Living Persons Privacy via Row Level Security (RLS)

Scenario: A distant user attempts to query personal contact info or sensitive biography of living relatives.

Engine Solution: PostgreSQL Row Level Security (RLS) policies mask phone numbers and full dates for nodes where `is_alive = true`, unless queried by immediate family or authorized stewards.